

Why Finance Still Doesn't Have a Proper Data Layer — And What an Ontology Can Actually Fix

Nyquist Research

11 May 2026

Abstract

An audit of the five semantic failure modes that consume 60–80% of every quant team's working time — identity, corporate actions, event taxonomy, cross-asset joins, temporal semantics — and the layered architecture that eliminates 80–90% of them at infrastructure cost, not compute cost.

Nyquist Research · 02 · Published 11 May 2026 · Reading time: 16 min · Topic: Data Infrastructure · Ontology · Standards referenced: FIBO · FIX · ACTUS · FIGI · LEI · RDF/DWL

Your data pipeline broke again — not because of the feed, but because vendor A calls it `adjusted_close`, vendor B calls it `close_adj`, and your intern called it `price`. This is not a naming problem. It is an **ontology** problem.

And here is the uncomfortable arithmetic: the global financial data services market reached \$29.5 billion in 2024 and is on track to exceed \$63 billion by 2032. Yet a significant portion of every quant team's working time is consumed not by alpha generation, but by identifier mapping, schema reconciliation, and corporate action normalisation. A pricing quant developer at a hedge fund once described it with brutal honesty: data retrieval, storage, cleaning, and normalisation was “about 80% of my job.” The alpha model — the intellectual product the firm actually pays for — was the remaining 20%.

This is not a resource problem. It is not a talent problem. It is a **structural** problem rooted in the absence of a shared semantic layer. And the solution has existed in computer science for decades. The financial industry has simply failed to adopt it at infrastructure scale.

1 Part I · Semantic fragmentation in financial data

Financial data exists in parallel semantic universes that were never designed to converge. Each failure mode is well understood in isolation. Together, they compound into a productivity catastrophe.

Mode 01 · Identity

No universal instrument schema.

Bloomberg FLDS, Refinitiv RIC, FIGI, CUSIP, ISIN, internal IDs — an instrument exists simultaneously under five or more identifiers with no formal cross-reference standard. G-Research publicly identifies “symbolology” as one of the hardest problems in their data engineering

stack. When identifier resolution logic is bespoke per vendor, errors become invisible: a join that silently drops 3% of instruments is indistinguishable from a correct join unless you have authoritative ground truth.

Mode 02 · Corporate actions

Inconsistency across every vendor.

Split factors, dividend treatment, spin-off handling, rights issue normalisation — all differ with no normative mapping. The same stock split on the same date can produce three different adjusted price series. The 0.1% discrepancy gets missed in code review. Over a 10-year backtest, it moves Sharpe by tenths. There is no standard specifying *how* adjusted prices must be calculated — every vendor makes defensible choices, just *different* choices.

Mode 03 · Event taxonomy

"Earnings date" has at least six legitimate answers.

Press release date, 8-K filing date, earnings call transcript date, preliminary results date, final audited filing date, or vendor processing timestamp. Datasets disagree — undocumented, because each vendor's definition is self-evidently correct to them. For event-driven strategies (PEAD, earnings momentum), this is not a data quality issue. It *is* the strategy. A one-day window error means you are backtesting something that does not correspond to any tradeable signal.

Mode 04 · Cross-asset joins

No common entity model across asset classes.

Equities, government rates, credit spreads, FX spots, commodity futures — each asset class built by a different vendor, for a different buyer, with a different internal schema. Joining macro rate data to equity issuer data requires bespoke engineering for every combination. Quant teams accumulate a growing catalogue of one-off join scripts — each with undocumented assumptions, each silently breaking when a source changes schema.

Mode 05 · Temporal semantics

Point-in-time, as-of, as-reported — routinely conflated.

Three distinct temporal concepts: what was *known* at a moment, what is *valid* for a date, what was *originally filed*. Most vendor feeds deliver a single timestamp whose meaning is encoded in database architecture, invisible at the API surface. A 2015 value-factor backtest that uses earnings revised in 2018 is testing a strategy that did not exist in 2015. This is the subtlest failure mode, and the most expensive.

1.1 The compounding scenario

Consider a quant PM at a mid-size macro fund attempting to backtest a cross-asset equity-rates strategy. In week one:

- The equity universe join fails because Bloomberg uses FIGI while the rates feed uses internal RIC codes — the mapping is not maintained systematically.

- The corporate actions table has split factors applied inconsistently between two equity vendors, producing price level discrepancies that look like data gaps.
- Earnings dates diverge by 1–3 days between sources due to pre-market vs. after-hours timestamp conventions.
- Macro indicator data uses final-revised figures, not real-time release vintage — GDP revisions introduce look-ahead bias into the macro signal.
- Rates data joins on instrument-level identifiers that have no formal link to the equity issuer entity model.

Five independent semantic failures. None produced a runtime error. All produced plausible-looking data. **The strategy that “works” in this backtest environment is not a trading strategy — it is an artefact of data engineering inconsistency.**

Bloomberg’s own survey of over 150 quants and data scientists confirmed that progress with AI in quantitative research “is increasingly shaped by data readiness rather than experimentation alone.” The firms with clean, semantically consistent data are pulling ahead. The rest are running faster on a treadmill.

2 Part II · What a financial ontology actually is — and what it isn’t

An **ontology**, in the W3C technical sense, is a formal specification of the concepts, relationships, and constraints within a domain — expressed in a machine-interpretable language. The two primary standards are:

RDF — Resource Description Framework

A graph-based data model where knowledge is represented as subject–predicate–object triples. The substrate.

OWL — Web Ontology Language

A vocabulary for defining classes, properties, restrictions, and axioms on top of RDF, enabling automated reasoning.

Not an ontology: a database schema (defines structure, not meaning) · a data dictionary (human-readable, not machine-inferable) · a taxonomy (hierarchy without properties or axioms) · a knowledge graph (a KG is the *populated instance* of an ontology — the ontology is the schema).

Analogy: the distinction between ontology and knowledge graph is precisely the distinction between a **class definition** and an **object instance** in OOP. Bloomberg operates one of the most sophisticated financial KGs in production — the KG contains the instances, the ontology defines what those instances *mean*.

2.1 Core components

Classes define the types of things that exist in the domain — `fin:Instrument`, `fin:Equity`, `fin:Bond`, `fin:Derivative`, `fin:CorporateAction`, `fin:PriceObservation`, `fin:LegalEntity`, `fin:ExchangeListing`, `fin:IndexMembership`.

Properties define relationships and attributes.

- *Object properties:* `fin:issuedBy`, `fin:listedOn`, `fin:adjustedBy`, `fin:memberOf`.

- *Datatype properties:* `fin:hasISIN`, `fin:hasFIGI`, `fin:observedAt`, `fin:closePrice`.

Axioms define the constraints that instances must satisfy. Cardinality (an Equity has at most one ISIN per jurisdiction). Domain/range (`fin:adjustedBy` requires a `fin:CorporateAction` as its range). Temporal validity (a price observation is valid only within its `valid_time` interval).

Below is a minimal but complete Equity instrument definition in Turtle/OWL — what AAPL *is*, what identifiers it carries, that adjusted prices are linked to specific corporate actions, and what constraints on cardinality must hold. **A database schema can encode the first two. Only an ontology encodes the third and fourth.**

Figure: Turtle · OWL 2, 35 lines, self-contained — Equity instrument definition

```
@prefix fin: <https://ontology.nyquist.io/finance#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix time: <http://www.w3.org/2006/time#> .

# -- Class hierarchy -----
fin:Instrument a owl:Class ;
    rdfs:label "Financial Instrument" .

fin:Equity a owl:Class ;
    rdfs:subClassOf fin:Instrument ;
    rdfs:label "Equity Instrument" .

fin:PriceTimeSeries a owl:Class ;
    rdfs:label "Price Time Series" .

fin:CorporateAction a owl:Class ;
    rdfs:label "Corporate Action" .

# -- Datatype properties -----
fin:hasISIN a owl:DatatypeProperty ;
    rdfs:domain fin:Instrument ;
    rdfs:range xsd:string ;
    owl:cardinality 1 ;
    rdfs:comment "ISIN - exactly one per instrument per jurisdiction" .

fin:hasFIGI a owl:DatatypeProperty ;
    rdfs:domain fin:Instrument ;
    rdfs:range xsd:string .

fin:hasRIC a owl:DatatypeProperty ;
    rdfs:domain fin:Instrument ;
    rdfs:range xsd:string .

# -- Object properties -----
fin:hasPriceSeries a owl:ObjectProperty ;
    rdfs:domain fin:Instrument ;
    rdfs:range fin:PriceTimeSeries .
```

```

fin:adjustedBy a owl:ObjectProperty ;
    rdfs:domain fin:PriceTimeSeries ;
    rdfs:range   fin:CorporateAction ;
    rdfs:comment "Links adjusted series to triggering CA" .

fin:issuedBy a owl:ObjectProperty ;
    rdfs:domain fin:Instrument ;
    rdfs:range   fin:LegalEntity .

# -- Instance data -----
fin:AAPL a fin:Equity ;
    fin:hasISIN   "US0378331005" ;
    fin:hasFIGI   "BBG000B9XRY4" ;
    fin:hasRIC     "AAPL.O" ;
    fin:hasPriceSeries fin:AAPL_AdjClose_USD .

fin:AAPL_AdjClose_USD a fin:PriceTimeSeries ;
    fin:priceType fin:AdjustedClose ;
    fin:currency  "USD" ;
    fin:adjustedBy fin:AAPL_SplitEvent_2020_08 .

```

What this encodes: class hierarchy + identifier carriers + adjustment provenance + cardinality constraints. A relational schema covers the first two slots. The third and fourth are what makes it an *ontology*, not a table.

3 Part III · Existing standards — and why they’re not enough

The industry has not been idle. Multiple standards address fragments of this problem. **The honest assessment is that each solves its assigned problem well and fails to integrate with any of the others.**

Table 1: Industry standards: coverage and core limitations

Standard	Owner	Coverage	Core limitation
FIBO v3.0	EDM Council	Broad financial concepts: instruments, entities, contracts, processes	Conceptually rich; <i>operationally immature</i> . OWL 2 DL reasoning over the full corpus fails consistency checks in standard reasoners due to complexity. Adoption gap between firms that <i>reference</i> FIBO and firms that <i>deploy</i> it is wide.

Standard	Owner	Coverage	Core limitation
FIX 5.0 SP2	FIX Trading Community	Trade and order message semantics, execution reporting	Messaging protocol only — defines <i>wire format and field tags</i> , not a persistent data model. No temporal semantics. No instrument-level model beyond what is needed for order routing.
ACTUS v1.0	ACTUS Research Foundation	Algorithmic representation of contract cash flows (PAM, ANN, LAM, ...)	Narrow scope by design: contract cash flow simulation. <i>No market data layer</i> . No cross-asset identity model. No event taxonomy. Excellent for what it does; irrelevant to data wrangling in quant research workflows.
FIGI / OpenFIGI	OMG · Bloomberg	Instrument identification	Identifier only — no properties, no relationships, no temporal validity. Mapping to ISIN/CUSIP/RIC requires external tooling and is not always current. <i>Useful as cross-reference anchor; insufficient as semantic model</i> .
LEI / GLEIF	GLEIF	Legal entity identity	Entity-level only. No linkage to instrument-level data. No index membership, no issuer → instrument graph, no counterparty relationship model. Required for MiFID II / EMIR reporting; <i>insufficient for quant data infrastructure</i> .

3.1 The integration gap

The critical observation is structural: each standard solves one slice precisely. **FIGI** identifies instruments. **LEI** identifies entities. **ACTUS** models cash flows. **FIX** transmits orders. **FIBO**

conceptualises everything. **Nothing links them.**

The gap between “we use FIBO as our data dictionary” — which many large banks genuinely do — and “our quant data stack is semantically consistent across vendors” is not a small implementation gap. It is an architectural chasm. FIBO’s own reasoning challenges are well-documented: even basic consistency checking of the full FIBO OWL corpus times out in standard DL reasoners. The ontology evolved from the 2008 financial crisis, when practitioners recognised the urgent need for a shared vocabulary for financial contracts. It has since grown into a comprehensive but operationally difficult artefact.

No existing standard is the answer. What exists is a collection of useful building blocks. The engineering task is to *assemble* them into a coherent operational layer — and to make that layer the *foundation* of the data infrastructure, not a retrospective mapping exercise.

4 Part IV · What a proper layer looks like in practice

A production-grade implementation does not require deploying full OWL 2 DL reasoning over a tick database. The architecture that eliminates 80–90% of wrangling is a layered semantic model with clear separation of concerns.

Five layers · bottom-up (from canonical identity to point-in-time semantics)

L5 · Temporal layer · bi-temporal model

`valid_time` × `transaction_time`. Point-in-time semantics enforced at every layer below. *Prevents look-ahead bias.*

L4 · Relationship layer · the cross-asset graph

Issuer → Instrument → Listing → Index → Portfolio. Traversable graph; multi-asset path queries instead of bespoke joins. *Enables multi-asset strategies.*

L3 · Market data layer · methodology as type

`PriceObservation` typed by methodology — VWAP · AdjustedClose · Settlement · TheoreticalMid. Methodology is a *class*, not a string. *Eliminates silent discrepancies.*

L2 · Event layer · canonical normalisation

`CorporateAction` · `EarningsEvent` · `IndexRebalance`. Normalised; `as_released` + `as_revised` variants with `valid_time` / `transaction_time` stamps. *Queryable by historical knowledge.*

L1 · Identity layer · the foundation everything rests on

ISIN ↔ FIGI ↔ RIC ↔ CUSIP ↔ LEI ↔ `internal_id`. Bidirectional cross-reference graph, canonical resolution. *Build this first or build nothing.*

4.1 L1 Identity — the foundation everything else rests on

This is a bidirectional cross-reference graph. Every instrument, every legal entity, every data vendor’s identifier maps to a single canonical internal node. **This graph is not a lookup table maintained in a spreadsheet. It is a typed property graph with versioning and temporal validity.**

The identity layer solves the most common point of pipeline failure: joining data from two sources that use different identifier schemas. By resolving to canonical nodes *before* any join

operation, the identity layer makes vendor-specific identifier differences invisible to downstream consumers.

Implementation choices: a property graph database (Neo4j, Memgraph) is the natural fit for this layer. Graph databases achieve constant-time relationship traversal, whereas relational databases executing equivalent JOIN operations degrade as graph depth increases. For identity resolution specifically — traversing N hops from an external identifier to a canonical node — property graphs are architecturally superior.

4.2 L2 Events — normalisation with temporal precision

Corporate actions are the most error-prone category of financial data. Every split, dividend, spin-off, and rights issue requires a chain of decisions: what is the effective date? what is the adjustment factor? does it apply to price, volume, or both? how are open orders affected?

The event layer normalises these decisions into a canonical representation with two versions: `as_released` (the event as first published) and `as_revised` (incorporating subsequent corrections). Both carry `valid_time` and `transaction_time` stamps. **A backtest can query for the event as it was known on any specific historical date — not as it appears today.**

4.3 L3 Market data — types, not names

The critical design decision in the market data layer is to treat price methodology as a *semantic type*, not a column naming convention. `AdjustedClose` is a class, not a string. It has properties: it was adjusted for which corporate actions, using which methodology, by which vendor.

This means that `fin:AAPL_AdjClose_BloombergMethod` and `fin:AAPL_AdjClose_RefinitivMethod` are distinct objects with a shared supertype. A query that asks “*give me the adjusted close for AAPL on 2019-08-01*” can explicitly specify which methodology it wants — or can request any, with the methodology recorded in the output. **No silent discrepancies. No implicit assumptions about what “adjusted” means.**

4.4 L4 Relationships — the cross-asset graph

The relationship layer is what makes multi-asset strategies tractable. When an instrument has a typed relationship to its issuer (`issuedBy`), its exchange listing (`listedOn`), its index memberships (`memberOf`), and its portfolio holdings (`includedIn`), cross-asset joins become graph traversals rather than bespoke engineering projects.

A query like “*give me all instruments in the S&P 500 as of 2020-01-01 whose issuer had a credit rating of BB or above on that date*” is a three-hop graph traversal. In a relational model with separate vendor tables for equity data, credit data, and index constituents, it is a custom join pipeline with manual date alignment.

4.5 L5 Bi-temporality — the difference between backtesting and fiction

The temporal layer is **not optional**. It is the mechanism by which the entire stack acquires point-in-time semantics. The bi-temporal model maintains two independent time axes for every record:

- `valid_time` — the period during which the fact was true in the real world.
- `transaction_time` — the period during which the fact was recorded in the system.

A fact with `valid_time = [2019-01-01, 2019-12-31]` and `transaction_time = [2020-03-15, ∞)` is a fact that was true during 2019 but only entered the database in March 2020. This is a restated earnings figure. Without bi-temporal modelling, a backtest run in 2021 would see this figure as if it had been available in 2019. **That is look-ahead bias.**

With bi-temporal modelling, a query with `AS OF TRANSACTION_TIME 2019-06-01` would not see this record at all. It did not exist in the system on that date. This is not a database feature request. It is an infrastructure requirement for any research-grade financial data system.

4.6 Worked example · ontology-aware identity resolution

The following demonstrates the practical difference between a naive vendor join and an ontology-aware cross-reference resolution. The identity graph uses `networkx` for clarity; a production implementation would use a property graph database or an `rdflib` RDF store.

Figure: Python · pandas + networkx, 80 lines · production sketch — ontology-aware identity resolution

```
import networkx as nx
import pandas as pd
from typing import Optional

# -- Build the identity resolution graph -----
# In production: load from your property graph DB (Neo4j, rdflib, etc.)
id_graph = nx.Graph()

# Apple Inc. - four identifier namespaces
id_graph.add_node("US0378331005", id_type="ISIN",      asset_class="Equity")
id_graph.add_node("BBG000B9XRY4", id_type="FIGI",      asset_class="Equity")
id_graph.add_node("AAPL.0",          id_type="RIC",      asset_class="Equity")
id_graph.add_node("037833100",       id_type="CUSIP",    asset_class="Equity")
id_graph.add_node("AAPL",            id_type="internal", asset_class="Equity")

id_graph.add_edges_from([
    ("US0378331005", "BBG000B9XRY4"),
    ("BBG000B9XRY4", "AAPL.0"),
    ("AAPL.0",       "037833100"),
    ("037833100",    "AAPL"),
])

def resolve_canonical(identifier: str, graph: nx.Graph) -> Optional[str]:
    """
    Walk the connected component of 'identifier' and return
    the node with id_type == 'internal'. Returns None if not found.
    """
    if identifier not in graph:
        return None
    component = nx.node_connected_component(graph, identifier)
    for node in component:
        if graph.nodes[node].get("id_type") == "internal":
            return node
```

```

    return None

# -- Simulate two vendor dataframes with different identifier schemas --
# Vendor A (Bloomberg): uses FIGI
prices_bbg = pd.DataFrame({
    "figi":      ["BBG000B9XRY4", "BBG000BVPV84"],
    "date":      ["2024-01-15",  "2024-01-15"],
    "close_adj": [185.20,        175.50],
    "volume":    [55_000_000,    40_000_000],
})

# Vendor B (Refinitiv): uses RIC
prices_ref = pd.DataFrame({
    "ric":      ["AAPL.0",      "MSFT.0"],
    "date":      ["2024-01-15", "2024-01-15"],
    "adjusted_close": [185.18,    175.48],
    "market_cap_usd": [2.9e12,    2.6e12],
})

# -- Resolve all identifiers to canonical internal IDs -----
prices_bbg["canonical"] = prices_bbg["figi"].map(
    lambda x: resolve_canonical(x, id_graph)
)
prices_ref["canonical"] = prices_ref["ric"].map(
    lambda x: resolve_canonical(x, id_graph)
)

# Drop rows where canonical resolution failed (unknown instruments)
prices_bbg = prices_bbg.dropna(subset=["canonical"])
prices_ref = prices_ref.dropna(subset=["canonical"])

# -- Join on canonical ID - vendor schemas are now irrelevant -----
merged = prices_bbg.merge(
    prices_ref[["canonical", "date", "adjusted_close", "market_cap_usd"]],
    on=["canonical", "date"],
    how="inner"
)

# -- Semantic validation: flag price divergence above threshold -----
merged["price_divergence_bps"] = (
    (merged["close_adj"] - merged["adjusted_close"]).abs()
    / merged["adjusted_close"] * 10_000
).round(1)

divergent = merged[merged["price_divergence_bps"] > 5.0]
if not divergent.empty:
    print(f"WARNING: {len(divergent)} instruments with price divergence > 5bps:")
    print(divergent[["canonical", "close_adj",
                    "adjusted_close", "price_divergence_bps"]])

print(merged[["canonical", "date", "close_adj", "adjusted_close",

```

```
"market_cap_usd", "price_divergence_bps"]])
```

Semantic validation as data quality signal: flags instruments where two vendor prices diverge by more than 5 basis points. In a naive `pd.merge()` approach, this discrepancy is invisible. In an ontology-aware pipeline, the mismatch surfaces *before* it reaches the model.

5 Part V · What actually gets fixed

5.1 Backtesting integrity

The most direct financial consequence of missing temporal semantics is **look-ahead bias** — when a backtest uses information that was not available to a market participant at the time of the simulated decision. It is the most common mistake in quantitative backtesting and the hardest to detect because it produces no runtime error — only inflated results.

Three mechanisms introduce look-ahead bias in financial data systems:

- **Restated fundamentals** — vendors overwrite original earnings figures with restated values, making the revised figure appear to have been available before the restatement was published.
- **Index additions backdating** — when vendors add a company to their database, they often include historical financial statements; a backtest using this database appears to have traded a company not in the original sample.
- **Vendor timestamp errors** — data timestamped at 9:30 am (market open) that was not actually delivered until 11:00 am introduces a 90-minute look-ahead window.

Survivorship bias alone inflates annual returns by 1.5–2.0% on S&P 500 backtests. For macro factor strategies using GDP revision data, using final-revised figures instead of real-time release vintages can produce systematic Sharpe inflation. **A point-in-time semantic layer makes this class of error structurally impossible:** a query with a given `transaction_time` cannot return records that were added to the system after that timestamp.

5.2 Pipeline reliability and observability

When semantic validation is embedded at the identity layer, a price observation without a typed methodology is invalid by schema definition. An instrument without a canonical ID cannot enter the pipeline. The failure mode shifts from “*silently wrong output*” to “*loud, early rejection*.”

This is the difference between data quality as a post-hoc audit exercise and data quality as an infrastructure guarantee. In production systems with daily data ingestion from 10+ vendors, the cost of silent errors compounds faster than the cost of loud failures. **Loud failures are fixed once. Silent errors are discovered during performance review.**

5.3 Cross-asset strategy development

When equities, rates, and FX instruments share a common ontological identity and a traversable relationship graph, multi-asset joins become path queries. Bloomberg’s internal knowledge graph was specifically designed to “*link data across different domains*” — enabling applications like news analytics where an article about a company can be linked to all instruments issued by that

company, all counterparty relationships, and all relevant economic indicators, through graph traversal rather than bespoke JOIN logic.

JPMorgan Chase has publicly referenced the value of knowledge graphs for understanding complex entity relationships and counterparty networks. The same architecture that enables fraud detection and regulatory compliance — a traversable graph of entity relationships — also enables cross-asset strategy construction. **The infrastructure investment is shared.**

5.4 Regulatory reporting as a by-product

MiFID II and EMIR already mandate LEI-linked transaction reporting. EMIR specifically requires transaction reports to include the LEI of the counterparty, the underlying instrument, and the relevant exchange — all linked semantic objects. Firms that maintain a proper LEI → instrument → counterparty graph generate these reports as a by-product of their normal data operations.

Firms without this infrastructure generate MiFID II reports by assembling ad-hoc joins from disparate systems, manually reconciled before each regulatory submission. Each new reporting requirement adds a new pipeline. **The semantic data layer converts regulatory compliance from a periodic firefighting exercise into a standing query.**

6 Part VI · What ontologies cannot easily fix

6.1 The adoption problem

The technical implementation of a financial ontology is tractable. The Python libraries are mature (`rdflib` handles OWL 2 RL reasoning, Apache Jena handles SPARQL endpoints, Neo4j handles property graph deployment). The standards exist. The tooling is available.

The hard problem is organisational. Converging four teams with four vendor contracts, four internal naming conventions, and four existing data pipelines on a shared semantic schema is not a technical challenge. It is a political one. Each team has legitimate local reasons to prefer their existing approach. The marginal cost of the transition is immediate and certain. The benefit is deferred and shared.

This is why the most successful implementations start small: one asset class, one data vendor pair, one identifiable pain point. *Prove value. Expand incrementally.*

6.2 Performance tradeoffs · SPARQL vs. property graph

Full SPARQL over an RDF triplestore is expressive and standards-compliant. It is **not fast** over tick-level data. A SPARQL query over 100 billion price observations is the wrong architecture.

The practical resolution is a hybrid: use the graph layer (property graph or RDF KG) for identity resolution, relationship traversal, and event normalisation — all relatively low-volume operations. Use a columnar time-series store (ClickHouse, Arctic/ArcticDB, kdb+) for price history retrieval. **The two layers communicate through the canonical identity system:** the KG resolves an instrument to a canonical ID; the columnar store retrieves the time series for that ID.

Graph databases achieve constant-time relationship traversal, while relational databases slow with complex JOIN operations — making property graphs architecturally superior for multi-hop identity resolution and relationship queries, and columnar stores superior for time-series

aggregations. *Separation of concerns is the answer, not choosing one technology for everything.*

6.3 OWL reasoning at scale

Full OWL 2 DL reasoning is computationally expensive. The FIBO corpus is large enough that standard DL reasoners time out on basic consistency checks. **This is not a reason to abandon semantic modelling — it is a reason to choose the right OWL profile.**

OWL 2 RL (Rule Language) is specifically designed for data-intensive applications where full DL expressiveness is not required. It supports Horn-clause reasoning, is implementable in standard rule engines, and scales to large triple stores. For a financial ontology that needs to enforce cardinality constraints, validate temporal consistency, and infer cross-identifier equivalences, OWL 2 RL is the correct profile. *OWL 2 DL is overkill for most production quant infrastructure use cases.*

6.4 Ontology maintenance

An ontology is not a one-time artefact. It is a living specification that must evolve with the domain. Every new asset class (private credit, digital assets, structured products), every new corporate action type (SPACs, dual-class conversions), and every new data vendor introduces maintenance obligations.

The discipline required is the same as software versioning: semantic versioning with explicit migration paths, a changelog, and backward-compatibility guarantees for downstream consumers. **Treat the ontology as a public API.** Break it deliberately and communicate the break, or maintain backward compatibility.

7 Part VII · Where Nyquist fits

In building Nyquist’s data layer, we confronted every one of these problems in sequence and in combination. Our approach was to build an **identity resolution graph at the core of the API layer** — so that regardless of which vendor feed a user queries, instruments, events, and prices resolve to a consistent semantic model.

It is not a full OWL ontology. A full OWL 2 DL deployment would be over-engineered for the use cases we currently serve. What we built is the *pragmatic 80% that eliminates 90% of the wrangling*: a typed identity graph with bidirectional cross-reference resolution, normalised corporate actions with point-in-time flags, and price observations typed by methodology rather than column name. **Semantic consistency is enforced at ingestion, not at query time, and certainly not at analysis time.**

The identity layer is the primitive that everything else depends on. Build it first. Build it well. The event layer, the relationship graph, and the temporal model are incrementally valuable *on top of* a solid identity foundation. They are expensive noise without one.

8 Key takeaways

The following five actions represent the highest-ROI starting points for quant teams and data engineers dealing with these problems today.

1. **Build the identity layer first — everything else depends on it.** Create a bidirectional cross-reference graph mapping all vendor identifiers (ISIN ↔ FIGI ↔ RIC ↔ CUSIP ↔ internal) to canonical internal nodes. This single artefact eliminates the majority of pipeline join failures and is reusable across every downstream system. Use a property graph database or a maintained mapping table — never a hardcoded join key in a notebook.
2. **Type your price observations as semantic classes, not column names.** `AdjustedClose`, `VWAP`, `SettlementPrice`, `TheoreticalMidpoint` are meaningfully different objects. Treat them as typed classes with explicit properties. Enforce this at ingestion. Trusting that `close_adj` and `adjusted_close` mean the same thing is a bet you will eventually lose.
3. **Implement bi-temporality for fundamentals before everything else.** Full bi-temporal modelling across all data types is expensive. The highest-impact starting point is earnings fundamentals and index constituent data, where look-ahead bias from restated figures and backdated additions is most severe. The architecture decision: *never overwrite a historical record*. Always insert new versions with new transaction timestamps.
4. **Use FIGI for instrument identity and LEI for entity identity — but own the bridge between them.** Neither standard links to the other. FIGI identifies instruments; LEI identifies entities; the relationship between an instrument and its issuer is yours to model and maintain. This bridge — `instrument hasIssuer entity` — is the join key for every cross-asset, regulatory, and counterparty exposure analysis. It is your infrastructure moat.
5. **Choose your semantic stack by use case, not by standard.** SPARQL + RDF for relationship traversal and constraint validation. Columnar time-series (ClickHouse, ArcticDB, kdb+) for price and volume history. Property graph (Neo4j, Memgraph) for high-throughput identity resolution. OWL 2 RL profile for inference — not OWL 2 DL. *The answer is a composed architecture, not a single technology.*

9 Constraints & limitations

Intellectual honesty about where this approach has real limits:

9.1 Full OWL ontology has high adoption cost

OWL 2 DL reasoning overhead and tooling complexity are not justified for most production quant pipelines. A pragmatic property graph with semantic typing conventions delivers most of the benefit at a fraction of the cost. *Do not deploy full OWL 2 DL unless you have a specific need for automated reasoning that cannot be served by simpler rule engines.*

9.2 SPARQL performance at tick-data scale is architecturally wrong

Separate the identity/relationship layer (property graph or RDF store) from the time-series retrieval layer (columnar store). SPARQL over billions of tick observations is the wrong tool. **Design accordingly from day one.**

9.3 Organisational buy-in is the harder problem

The technical implementation is the easy part. Converging multiple teams with established vendor relationships and local naming conventions on a shared semantic schema requires executive

sponsorship, clear ownership of the ontology as a shared infrastructure primitive, and incremental proof of value. *Start with one asset class. Prove it. Expand.*

9.4 Ontology maintenance burden scales with coverage

Every new asset class, data vendor, and corporate action type adds schema maintenance. Version the ontology explicitly. Treat breaking changes as API breaking changes. **Establish a governance process before the ontology grows beyond the point where a single person can maintain it coherently.**

9.5 Knowledge graph query performance tradeoffs are real

Property graph traversal outperforms relational JOIN at depth, but underperforms columnar stores for aggregations. The hybrid architecture described above is not a compromise — it is the correct design. *Evaluate each component against its specific query pattern.*

10 Closing

The relevant question for quant practitioners in 2026 is not whether to invest in a semantic data layer — the firms with clean, ontology-aware data are already pulling ahead of the rest, structurally and visibly. The question is whether the investment is disciplined:

- Is the identity layer first, or bolted on retroactively?
- Are price observations typed by methodology, or by column name?
- Is bi-temporality enforced for fundamentals, or treated as a nice-to-have?
- Is the ontology versioned like an API, or maintained by tribal knowledge?

A naming problem is fixed once. An ontology problem compounds until you fix the architecture.

How much of your team's time goes to identifier mapping and data normalisation?

We are curious whether the 60–80% estimate holds across firm sizes — from two-person quant shops to multi-strategy platform funds. And more specifically: where does the time actually go — identity layer, temporal semantics, corporate actions, or something else entirely?

If you are building a data layer for multi-asset strategies and hitting identity resolution walls — [we have been there. Let's compare notes.](#)